2026 PDS Mole Patrol Repository Summary

Technical repository summary

Project Overview

This repository contains the second semester Projects in Data Science project for The Mole Patrol. The project uses the PAD-UFES-20 smartphone skin lesion dataset to build a binary cancer classifier. The classifier predicts whether a lesion image belongs to a cancer class or a non-cancer class.

The original PAD-UFES-20 labels contain six diagnoses:

- Cancer: `BCC`, `SCC`, `MEL`
- Non-cancer: `ACK`, `NEV`, `SEK`

The code reduces these six classes to a binary target in `src/cancer.py`, where `BCC`, `SCC`, and `MEL` are mapped to `1`, and all other diagnoses are mapped to `0`.

The project is built around interpretable ABC-style lesion features:

- `A`: asymmetry of the lesion mask
- `B`: border compactness of the lesion mask
- `C`: colour statistics from lesion pixels

The final implemented model family is a Random Forest consensus model. Each model variant is made from five fold-trained `RandomForestClassifier` submodels, and the final prediction is decided by majority vote across those five submodels.

Repository Layout

Important paths:

- `README.md`: original project template and hand-in structure.
- `main.py`: trains or loads the eight model variants and evaluates them.
- `src/main_extraction.py`: main feature extraction pipeline.
- `src/feature_A.py`: asymmetry feature extraction.
- `src/feature_B.py`: border compactness feature extraction.
- `src/feature_C.py`: HSV, RGB, and greyscale colour feature extraction.
- `src/shortcuts.py`: hair and pen-mark shortcut removal.
- `src/traintestsplit.py`: validation holdout removal and five-fold grouped stratified train/test splitting.
- `src/basemodeltrain.py`: random forest training logic.
- `src/validation.py`: five-model majority-vote validation.
- `src/export_predictions.py`: exports saved model predictions to CSV.
- `data/metadata.csv`: PAD-UFES-20 metadata.

- `data/baseline_features.csv`: extracted baseline feature table.
- `data/extended_features.csv`: extracted extended feature table.
- `data/Validation.csv`: manually reserved validation patients/images.
- `data/annotations_combined.csv`: hair and pen-mark annotation votes.
- `data/imgs/` and `data/masks/`: expected raw image and mask folders. The actual images are not committed.
- `results/models/`: saved `.pkl` model arrays.
- `results/predictions/`: saved prediction CSVs.
- `results/figures/`: report figures.
- `results/reports/PDS_2026_MOLE_PATROL_REPORT.pdf`: final report.

Data Starting Point

The repository metadata contains:

- 2,298 image rows
- 1,373 unique patients
- Diagnosis counts:
  - BCC: 845
  - ACK: 730
  - NEV: 244
  - SEK: 235
  - SCC: 192
  - MEL: 52

The feature extraction pipeline excludes images that do not have a usable image/mask pair. A usable row must have:

- an image file in `data/imgs/`
- a corresponding mask file in `data/masks/`
- a readable image and mask
- a non-empty mask
- matching image and mask dimensions

After these checks, both generated feature tables contain:

- 2,095 image rows
- 1,288 unique patients

This means 203 metadata rows were excluded because they did not pass the usable image/mask checks.

Step 1: Feature Extraction

Feature extraction is coordinated by `src/main_extraction.py`.

For each usable image row, the pipeline:

1. Loads the image and its binary lesion mask.
2. Normalizes the image id by removing file extensions.
3. Extracts shape features from the mask.
4. Optionally applies hair and pen-mark removal to the image.
5. Extracts colour statistics from pixels inside the lesion mask.
6. Writes the extracted features to CSV.

The two main output files are:

- `data/baseline_features.csv`
- `data/extended_features.csv`

Asymmetry

Implemented in `src/feature_A.py`.

The asymmetry score is based on comparing the lesion mask with a mirrored version of itself. The code:

1. Converts the mask to binary values.
2. Finds the vertical midpoint that splits the lesion area in half.
3. Crops around that midpoint.
4. Compares the crop to its left-right mirror using logical XOR.
5. Normalizes the difference by lesion area.
6. Repeats the process over six rotations of 30 degrees.
7. Returns the average asymmetry score.

A higher score means the lesion shape is less symmetric.

Border Compactness

Implemented in `src/feature_B.py`.

The border feature uses the Polsby-Popper compactness formula:

```
compactness = (4 * pi * area) / perimeter^2
```

The code estimates the lesion perimeter by eroding the binary mask and subtracting the eroded mask from the original mask. A score closer to `1` indicates a more circular or compact lesion; a score closer to `0` indicates a more irregular border.

Colour Features

Implemented in `src/feature_C.py`.

The colour feature extractor keeps only pixels inside the lesion mask and computes summary statistics.

For HSV, the code calculates these statistics for hue, saturation, and value:

- mean
- standard deviation
- skew
- 5th percentile
- 50th percentile
- 95th percentile

For RGB and greyscale, the same statistics are calculated for:

- red
- green
- blue
- greyscale

Greyscale is computed as:

```
0.2989 * red + 0.5870 * green + 0.1140 * blue
```

Step 2: Shortcut Removal

Shortcut removal is implemented in `src/shortcuts.py`.

The purpose is to reduce visual artifacts that could distort colour features, especially:

- hair
- pen marks

The shortcut code uses `data/annotations_combined.csv`, which contains 1,637 annotated images with five hair votes and five pen votes per image. At least four valid votes are required before trusting the annotation.

Hair handling:

- Hair votes are averaged.
- Scores are converted into `skip`, `moderate`, or `heavy`.
- If brushing is needed, the image is converted to greyscale, blurred, processed with a black-hat morphological filter, thresholded, cleaned, and dilated.

Pen handling:

- Pen votes are averaged into present/absent.
- If pen removal is needed, the image is converted to HSV and thresholded for pen-coloured pixels.
- The pen mask is closed and dilated.

The hair and pen masks are combined, then OpenCV Telea inpainting fills the marked pixels. The cleaned image is used for colour extraction in the extended pipeline. Shape features still come from the original lesion mask.

Step 3: Baseline and Extended Feature Sets

The project distinction is:

- Baseline: asymmetry, border, and HSV colour statistics.
- Extended: the same general ABC structure, but with shortcut removal and additional RGB plus greyscale statistics.

The generated CSV files reflect that intention:

- `baseline_features.csv` contains patient id, diagnosis, image id, asymmetry, border, and HSV statistics.
- `extended_features.csv` contains patient id, diagnosis, image id, asymmetry, HSV statistics, RGB statistics, greyscale statistics, and border.

In the saved model naming, `extendedmodel.pkl` is the brushed HSV-only extended model. The full HSV + RGB + greyscale model is saved separately as `allcolormodel.pkl`.

Step 4: Validation Holdout and Train/Test Splitting

The split logic is implemented in `src/traintestsplit.py`.

The project had two competing requirements:

- preserve diagnosis balance through stratification
- prevent patient-level data leakage by keeping all images from the same patient in the same split

The final implementation uses a precomputed validation file plus scikit-learn grouped cross-validation:

1. `data/Validation.csv` lists validation images/patients.
2. `spliting(df, valdf)` reads the validation patient ids.
3. All validation patients are removed from the metadata before training/testing.
4. The remaining data is split using `StratifiedGroupKFold`.
5. `patient_id` is used as the group, so images from the same patient should not be split across train and test folds.
6. `diagnostic` is used for stratification.
7. Five folds are created with `shuffle=True` and `random_state=11037`.

Current validation file size:

- 440 validation rows
- 222 unique validation patients
- 208 cancer rows and 232 non-cancer rows in `data/Validation.csv`

Relative to the full metadata file, the validation CSV is about 19.15% of rows. The remaining metadata is about 80.85% of rows. Each fold then trains on roughly four fifths of the remaining data and tests on roughly one fifth of the remaining data.

Because the prediction exports operate only on rows that also exist in the extracted feature tables, the saved validation prediction CSVs contain 401 rows rather than 440.

The reason for reserving the validation set separately is that the group and stratification requirements make the split more complicated than a normal `train_test_split`. A simple random split can preserve class balance, but it can place images from the same patient in both training and validation. That would leak patient-specific information into evaluation. `StratifiedGroupKFold` solves the grouped stratified fold problem, but it does not directly create a separate untouched validation holdout before cross-validation. The repository therefore removes the validation patients first, then applies grouped stratified folds only to the remaining training/testing pool.

Step 5: Model Training

Model training is implemented in `src/basemodeltrain.py`.

Each model variant is trained as an array of five submodels:

- one submodel per grouped stratified fold
- each submodel is a `RandomForestClassifier`
- `n_estimators=200`
- `max_depth=4`
- `random_state=11037`

The five submodels are saved together as one `.pkl` file per model variant.

The phrase "consensus model" in this repository means that the five fold-trained random forest models vote on the final prediction. In `src/validation.py` and `src/export_predictions.py`, the final class is `1` when at least three of the five models predict `1`; otherwise it is `0`.

Step 6: The Eight Model Variants

The repository trains and saves eight model variants. They differ by which colour feature groups are included.

Model name	Saved model	Prediction file	Feature source	Colour features used	Saved validation accuracy	Saved hard-label AUC
Base	`basemodel.pkl`	`predictions_base.csv`	`baseline_features.csv`	HSV baseline feature set	0.5661	0.5656
Extended HSV	`extendedmodel.pkl`	`predictions_extended.csv`	`extended_features.csv`	HSV	0.7257	0.7250
Grey	`greymodel.pkl`	`predictions_grey.csv`	`extended_features.csv`	greyscale	0.6234	0.6229
RGB	`rgbmodel.pkl`	`predictions_rgb.csv`	`extended_features.csv`	RGB	0.6683	0.6686
RGB + Grey	`rgbgreymodel.pkl`	`predictions_rgb_grey.csv`	`extended_features.csv`	RGB and greyscale	0.6584	0.6586
HSV + Grey	`hsvgreymodel.pkl`	`predictions_hsv_grey.csv`	`extended_features.csv`	HSV and greyscale	0.7382	0.7376
RGB + HSV	`rgbhsvmodel.pkl`	`predictions_rgb_hsv.csv`	`extended_features.csv`	RGB and HSV	0.7232	0.7226
All Colour	`allcolormodel.pkl`	`predictions_all.csv`	`extended_features.csv`	HSV, RGB, and greyscale	0.7207	0.7203

The AUC values above are calculated from hard predicted labels, matching the way `src/validation.py` uses `metrics.roc_auc_score(y, predictions)`. They are not

probability-based ROC-AUC values.

The best saved validation result is the HSV + Grey model, with accuracy `0.7382` and hard-label AUC `0.7376`. The extended HSV-only model is close behind, with accuracy `0.7257` and hard-label AUC `0.7250`.

Step 7: Prediction Export

The saved prediction CSVs in `results/predictions/` are produced by `src/export_predictions.py`.

For each model variant, it:

1. Loads the saved `.pkl` model array from `results/models/`.
2. Loads the relevant feature table.
3. Converts diagnosis labels to binary labels.
4. Selects validation rows using the patient ids from `data/Validation.csv`.
5. Selects the feature columns expected by the saved model.
6. Runs all five submodels.
7. Applies majority vote.
8. Writes a CSV containing:
    - `img_id`
    - `patient_id`
    - `true_label`
    - `predicted_label`

Step 8: Report and Visual Outputs

The report and figures live under `results/`.

Notable figure outputs include:

- diagnosis distribution plots
- images-per-patient distribution
- feature extraction pipeline diagram
- shape feature analysis
- colour feature visualisation
- shortcut processing examples
- model performance plot
- tree visualisations from earlier model experiments

The report concludes that:

- binary classification worked better than the original six-class problem
- tree-based models were a reasonable choice because they do not require feature scaling

- random forests improved over single decision trees
- HSV colour features were generally stronger than RGB-only features
- adding greyscale to HSV gave the best saved validation result, though the improvement over HSV-only was small
- melanoma under-representation remains a major limitation
- the evaluation is internal only and would need external validation for stronger claims

Technical Notes

These are useful when reading or modifying the repository:

1. The model should be described as a Random Forest consensus model.
2. `extendedmodel.pkl` is the brushed HSV-only extended model.
3. `allcolormodel.pkl` is the full HSV + RGB + greyscale model.
4. `main.py` computes validation metrics but does not write prediction CSVs itself. Prediction CSV export is handled by `src/export_predictions.py`.
5. `main.py` has a `load_model=True` path, but the feature DataFrames are prepared only in the training branch, so that path may need adjustment before it can validate loaded models directly.
6. The validation set has 440 rows, but the saved prediction CSVs have 401 rows because only validation rows with extracted features are predicted.
7. The AUC values in the saved workflow are based on binary predicted labels, not predicted probabilities.